Phishing Shield AI — Technical Stack & Architecture

Simple & Clear Overview of Technologies, AI Models, and Cybersecurity Protection

Author / System Architect: Ram Narayan | **Framework:** Python FastAPI + React + Scikit-Learn | **Date:** October 2026

---

1. What is Phishing Shield AI?

Phishing Shield AI is an AI-powered cybersecurity tool that protects users from online scams and phishing attacks in real time.

When you receive a suspicious email, message, or website link, the system scans it immediately and tells you:

1. **Safety Score (0 to 100)**: How dangerous the message or link is.
2. **Clear Verdict**: Whether it is **SAFE**, **SUSPICIOUS**, or **CRITICAL (DANGEROUS)**.
3. **Simple Explanation**: Exactly why it is dangerous (e.g., fake banking link, urgent money demand, unknown phone number).

---

2. Simple How-It-Works Flow

Instead of complex diagrams, here is the simple 4-step workflow:

Step 1: Inbound Message / Link

A user copies a message into the Web Dashboard, or the Chrome Extension scans a webpage automatically.

Step 2: API Perimeter Check (FastAPI Backend)

The message is sent safely to our Python FastAPI backend server. Oversized files and spam traffic are blocked instantly.

Step 3: Multi-Layer AI & Rule Inspection

Four specialized checkers inspect the message at the same time:

- **Text AI Model**: Reads the words to detect panic, urgency, or fake lottery/refund claims.
- **URL / Link Checker**: Checks if the website address is fake (e.g., `paypa1.com` pretending to be `paypal.com`).
- **Phone Number Checker**: Checks if an unknown personal mobile number is asking for banking passwords or OTPs.
- **Email Checker**: Checks if official-looking emails are actually coming from free fake accounts (like `@gmail.com`).

Step 4: Real-Time Warning & Storage

The system shows a color-coded warning alert to the user and securely saves the threat record into MongoDB without storing private personal messages.

---

3. Technology Stack Breakdown (What Technologies Are Used & Why)

3.1. Frontend (User Interface)

- **React 18**:
- *What it is*: A popular JavaScript library for building responsive web pages.
- *Why we use it*: It updates the live security dashboard instantly whenever a new threat is detected without refreshing the page.
- **Vite 5**:
- *What it is*: A fast modern frontend build tool.
- *Why we use it*: Loads the website instantly during development and creates small, super-fast production files.

- **Tailwind CSS**:
- *What it is*: A modern styling framework.
- *Why we use it*: Gives the dashboard a clean, dark-mode cybersecurity theme with responsive cards, badges, and alerts.
- **Lucide Icons**:
- *What it is*: Clean, lightweight SVG icons.
- *Why we use it*: Shows visual icons like Shield, Alert, Lock, and Checkmark for quick threat recognition.

3.2. Backend (API & Server)

- **Python (v3.11)**:
- *What it is*: The main backend programming language.
- *Why we use it*: Python is the industry standard for AI, machine learning, and data processing.
- **FastAPI**:
- *What it is*: A fast, modern web framework for Python.
- *Why we use it*: Handles multiple incoming scan requests in milliseconds using asynchronous speed. It also automatically generates interactive API documentation (`/docs`).
- **Uvicorn**:
- *What it is*: A high-speed web server for Python FastAPI.
- *Why we use it*: Runs the backend smoothly in both local testing and cloud production environments.
- **Pydantic**:
- *What it is*: A data validation library.
- *Why we use it*: Ensures every incoming message has the correct format and blocks malicious input before it reaches the database.

3.3. Machine Learning & AI Models

- **Scikit-Learn**:
- *What it is*: The most trusted machine learning library in Python.
- *Why we use it*: Powers our text classification model with fast, reliable inference (under 15 milliseconds).
- **TF-IDF Vectorizer (Term Frequency - Inverse Document Frequency)**:
- *What it does in simple words*: Converts words and sentences into numbers that the computer can analyze. It gives higher importance to rare scam words (like "lottery", "suspended", "verify KYC", "urgent action") and ignores normal common words (like "the", "is", "at").
- **Logistic Regression Classifier**:
- *What it does in simple words*: A high-accuracy classification algorithm that looks at the numbers from TF-IDF and calculates the probability (0% to 100%) that the message is a phishing scam.
- **Training Dataset (29,113 Verified Samples)**:
- Trained on real-world datasets from Kaggle, corporate emails (Enron dataset), and SMS scam records.
- Achieves **96.84% accuracy** in detecting phishing emails and scam text messages.

3.4. Specialized Security Detection Engines

- **URL & Link Threat Engine (`url_detector.py`)**:
- Detects fake lookalike links (typosquatting), like `paypa1.com` instead of `paypal.com`.

- Flags insecure websites using plain `http://` instead of secure `https://`.
- Catches raw numerical IP addresses in links (e.g., `http://192.168.1.1/login`).
- **Phone & SMS Scammer Detector (`phone_detector.py`)**:
- Catches "Call this number immediately" fraud messages.
- Detects international scam country codes and fake callback traps.
- Flags when a message claims to be an official bank but gives an unofficial 10-digit personal mobile number.
- **Email Sender Verifier (`email_detector.py`)**:
- Flags scammers pretending to be banks or organizations using disposable burner emails or free consumer email accounts.
- **Hybrid Risk Arbitration Engine (`risk_engine.py`)**:
- Combines the scores from the AI model, the link checker, and the sender checker into a single final score between 0 and 100 with clear explanations.

3.5. Database & Data Privacy

- **MongoDB Atlas**:
- *What it is*: A scalable cloud database.
- *Why we use it*: Stores threat history, detection scores, and timestamps for security auditing.
- **Zero-PII Privacy Protection**:
- *What it means*: The database **does not** store personal message text or private passwords.
- *How*: It converts messages into a cryptographic **SHA-256 digital fingerprint** (hash) for deduplication so privacy is 100% protected.

3.6. Browser Extension

- **Chrome Extension (Manifest V3)**:
- *What it is*: An add-on for Google Chrome.
- *Why we use it*: Protects users while browsing the web by scanning webpage content and links before the user clicks on them.

---

4. Summary Table of Technologies Used

Category	Technology	Purpose (In Simple Words)
Frontend UI	React 18	Interactive live security dashboard
Frontend Tooling	Vite 5	Fast build tool and website compiler
Design / Styling	Tailwind CSS	Clean dark-mode cybersecurity theme
Backend Language	Python 3.11	Core logic, AI models, and data processing
Backend Framework	FastAPI	Fast web server for message scanning requests
Machine Learning	Scikit-Learn	Machine learning training and live predictions
Text Processing	TF-IDF Vectorizer	Translates message words into numerical features
AI Classifier	Logistic Regression	Calculates probability (0-100%) of phishing
Database	MongoDB Atlas	Stores threat alerts and scan logs safely
Browser Extension	Chrome Manifest V3	Real-time scanner inside Google Chrome browser

---

5. What Real Cybersecurity Attacks Does It Stop?

1. **Fake Bank & Account Suspension Scams**:

- *Example*: *"Your bank account will be blocked today! Click here to update KYC."*
- *How we stop it*: The NLP AI detects panic words, and the URL engine flags fake unverified domains.

2. **Typosquatting & Lookalike Domains**:

- *Example*: `netflix-billing-update.com` or `amz0n-verify.com`
- *How we stop it*: The URL engine spots brand impersonation and flags suspicious domain endings.

3. **Lottery & Refund Fraud**:

- *Example*: *"Congratulations! You won 25 Lakhs. WhatsApp our manager at +92..."*
- *How we stop it*: The phone engine identifies international fraud corridors and lottery scam triggers.

4. **Website Link Traps in Browser**:

- *Example*: Malicious hidden links on web pages.
- *How we stop it*: The Chrome Extension scans links right on the page and warns users before they click.